Scaffold FYP — Deep Dive: Papers, Technical Feasibility & Implementation Roadmap

PART 1: All Important Paper Links

Core LLM Tutoring Systems (What Already Exists)

Paper	Venue	Year	Link
CodeHelp: LLMs with Guardrails for Programming Classes	Koli Calling	2023	https://arxiv.org/abs/2308.06921
CodeAid: Balancing Student and Educator Needs	CHI	2024	https://arxiv.org/abs/2401.11314
STAP: Socratic Tutor for Adaptive Programming	ISAIE	2025	https://dl.acm.org/doi/10.1145/3775073.3775165
ClassAid: Real-time Instructor-AI-Student Orchestration	CHI	2026	https://dl.acm.org/doi/10.1145/3772318.3790824

Closing the Loop: Instructor-in-the-Loop AI for Programming	SIGCSE TS	2026	https://arxiv.org/abs/2510.14457

Guardrails & Answer Leakage Prevention

Paper	Venue	Year	Link
CodeGuard: Improving LLM Guardrails in CS Education	EACL Findings	2026	https://arxiv.org/abs/2602.02509
Evaluating Answer Leakage Robustness of LLM Tutors	ACL	2026	https://arxiv.org/abs/2604.18660
Mitigating Scaffolding Collapse in Socratic Tutors	arXiv	2026	https://arxiv.org/abs/2607.19371
EduGuardBench: Pedagogical Fidelity & Adversarial Safety	AAAI	2026	https://ojs.aaai.org/index.php/AAAI/article/view/40399
Evaluating Prompt Injection Defenses for Educational LLM Tutors	arXiv	2025	https://arxiv.org/abs/2605.06669

Knowledge Tracing for Programming

Paper	Venue	Year	Link
Deep Knowledge Tracing (original DKT)	NeurIPS	2015	https://arxiv.org/abs/1506.05908

Code-DKT: Code-based Knowledge Tracing	EDM	2022	https://arxiv.org/abs/2206.03545
srcML-DKT: Enhanced DKT with Code Representations	EDM	2025	https://educationaldatamining.org/EDM2025/proceedings/2025.EDM.short-papers.83/
DPKT: Difficulty-aware Programming KT via LLMs	Nature Sci. Reports	2025	https://www.nature.com/articles/s41598-025-96540-3
Open-Ended Knowledge Tracing (OKT)	EMNLP	2022	https://aclanthology.org/2022.emnlp-main.254/
pyBKT: Bayesian Knowledge Tracing Library	EDM	2021	https://arxiv.org/abs/2105.00385
Detecting Struggling Programmers using Proficiency Taxonomies	arXiv	2025	https://arxiv.org/abs/2508.17353
Automated KC Generation and Interpretable KT	arXiv	2025	https://arxiv.org/abs/2502.18632
Denoising Programming KT with Code Graphs	arXiv	2025	https://arxiv.org/abs/2506.11107

IDE Telemetry & Student Monitoring

Paper	Venue	Year	Link
CodeWatcher: IDE Telemetry for VS Code	ICSME	2025	https://arxiv.org/abs/2510.11536
Spark: Real-Time Monitoring of Programming Exercises	arXiv	2026	https://arxiv.org/abs/2601.22256
RECAP: Capturing & Replaying AI-Assisted Programming	arXiv	2025	https://arxiv.org/abs/2605.01104

Hint Generation & Scaffolding Theory

Paper	Venue	Year	Link
Navigating the Landscape of Hint Generation Research (survey)	arXiv	2024	https://arxiv.org/abs/2404.04728
Next-Step Hint Generation for Intro Programming	ACE	2023	https://arxiv.org/abs/2312.10055
One Step at a Time: LLMs + Static Analysis for Hints	Koli Calling	2024	https://dl.acm.org/doi/10.1145/3699538.3699556
Adaptive vs Planned Metacognitive Scaffolding	Computers & Education	2025	https://www.sciencedirect.com/science/article/abs/pii/S0360131525002416

Scaffolding Metacognition in Programming Education	arXiv	2025	https://arxiv.org/abs/2511.04144
A Theory of Adaptive Scaffolding for LLM-based Pedagogical Agents	arXiv	2025	https://arxiv.org/abs/2508.01503
Data-Driven Hints in Intelligent Tutoring Systems (survey)	arXiv	2025	https://arxiv.org/abs/2603.07311
Rethinking Scaffolding in LLM Tutors: Benchmarks vs Reality	arXiv	2026	https://arxiv.org/abs/2606.15766

LLM Pedagogical Alignment

Paper	Venue	Year	Link
When LLM Tutoring Responses Work: Evidence from Conversations	arXiv	2026	https://arxiv.org/abs/2607.09919
Playing Dumb to Get Smart: LLM Teachable Agent	CHI	2025	https://dl.acm.org/doi/10.1145/3706598.3713644

Improving Student-AI Interaction Through Pedagogical Prompting	arXiv	2025	https://arxiv.org/abs/2506.19107

Public Datasets

Dataset	Description	Link
CSEDM 2021 Data Challenge	130K+ code submissions, 800+ students, 50 problems, ProgSnap2 format	https://sites.google.com/ncsu.edu/csedm-dc-2021/
ProgSnap Toolkit	Tools + configs for loading public CS education datasets	https://github.com/CSSPLICE/ProgSnapToolkit
ASSISTments	Classic KT benchmark dataset (math, but useful for KT model dev)	Via pyBKT built-in loader

Open Source Tools & Libraries

Tool	What It Is	Link
pyBKT	Python Bayesian Knowledge Tracing library (pip install)	https://github.com/CAHLR/pyBKT
DKT PyTorch Collection	Multiple KT models (DKT, DKT+, SAKT, DKVMN, GKT, KQN) in PyTorch	https://github.com/hcnoh/knowledge-tracing-collection-pytorch

DKT+	DKT with prediction-consistent regularization	https://github.com/ckyeungac/deep-knowledge-tracing-plus
Dialogue-KT	LLM-based knowledge tracing from dialogues (LAK 2025)	https://github.com/umass-ml4ed/dialogue-kt
Gen-Ed (CodeHelp framework)	Mark Liffiton's open-source framework powering CodeHelp	https://github.com/liffiton/Gen-Ed
NeMo Guardrails	NVIDIA's open-source guardrail toolkit for LLMs	https://github.com/NVIDIA-NeMo/Guardrails
CodeGuard Dataset	8,000 prompts for CS education guardrail training	Referenced in paper, check https://arxiv.org/abs/2602.02509

PART 2: Can These Actually Be Combined Into One Product?

Short answer: Yes, and that's precisely why this is a good FYP.

Every component listed above is real, publicly available, and has been implemented independently. The gap is that nobody has wired them together. Here's the honest status of each:

What's Real and Usable Right Now

Knowledge Tracing: pyBKT is a `pip install pyBKT` away. It works with any tabular data in the format (student_id, skill_id, correct/incorrect). You can have a working BKT model predicting student mastery within a day. The PyTorch DKT collection gives you 6+ model variants ready to train. These are not theoretical — they're running code with example scripts.

LLM Hint Generation: You don't need to fine-tune anything. The Gen-Ed framework (CodeHelp's actual codebase) is open source Python/Flask. It shows you exactly how to build an LLM wrapper with guardrails for education. You can replicate their approach with any API-accessible model (Claude, GPT-4, or local open-weight models like Llama 3, Qwen, CodeLlama).

IDE Telemetry: Monaco Editor (VS Code's web editor) is open source and embeddable in any web app. You can capture every keystroke, edit, compilation event through its API. This is standard web development — no research prototype needed.

Guardrails: NVIDIA NeMo Guardrails is production-grade open source. CodeGuard's PromptShield is a fine-tuned sentence-encoder. Both are usable.

What Needs to Be Built (Your Contribution)

The integration layer — the "glue" — is what doesn't exist:

1. A telemetry pipeline that converts raw IDE events into struggle signals in real time
2. A bridge that feeds those signals (plus submission results) into a KT model that updates in real time
3. A prompt construction module that takes the KT model's output (concept mastery vector) and injects it into the LLM's context for hint generation
4. A disclosure policy engine that decides which hint level to use based on KT state + struggle type
5. An output filter that checks the LLM's response for code leakage before showing it to the student

This is ~5 interconnected services. That is exactly right for a 4-person FYP team over a year.

What's Only in Papers (Not Available Publicly)

- **Code-DKT** (Shi et al.): The paper describes the model, but there

is no official public repo. However, the architecture is fully described in the paper, and you can reimplement it using the public DKT PyTorch collection as a base. The CSEDM dataset they used IS public.
- **ClassAid**: No public code. It's a CHI 2026 paper — the system was built for their specific classroom. But you don't need their code; you need to build your own system that goes beyond what they did (automated KT instead of manual instructor switching).
- **STAP**: No public code, only a 4-page paper. But the four-stage pipeline (Check, Correct, Complement, Segment) is described clearly enough to reimplement.
- **srcML-DKT**: Code availability not confirmed, but srcML itself is a freely available tool, and DKT is open source.

Bottom line: You are not blocked by any unavailable component. Everything you need to start building exists as open-source software.

PART 3: How to Build the Knowledge Tracing Model

This is the most important technical question. Here's the practical path:

Option A: Start Simple with BKT (Recommended for Month 1–2)

Bayesian Knowledge Tracing is a Hidden Markov Model. For each student-skill pair, it estimates:

- $P(L_0)$: probability the student knew the skill initially
- $P(T)$: probability the student learns the skill after an opportunity
- $P(G)$: probability the student guesses correctly without knowing
- $P(S)$: probability the student slips (answers wrong despite knowing)

```
# This is literally how simple it is to start
```

```
pip install pyBKT

from pyBKT.models import Model
model = Model()
model.fit(data=your_dataframe)
# DataFrame needs: student_id, skill_name, correct (0/1)
predictions = model.predict(data=test_dataframe)
```

Why start here: It works, it's interpretable (you can see exactly what the model thinks), and it gives you a baseline to beat with more complex models.

How to adapt for your use case:

Your data format from the lab will look like:

```
student_id | problem_id | concept_tag | correct | timesta
S001       | P03        | recursion   | 0       | 14:32:0
S001       | P03        | recursion   | 0       | 14:45:1
S001       | P03        | recursion   | 1       | 14:58:3
```

You need to define the concept_tags (Knowledge Components). For a Data Structures course, this might be: arrays, linked_lists, stacks, queues, recursion, trees, binary_search, sorting, hashing, graphs — roughly 10–15 concepts. Tag each lab problem with 1–3 relevant concepts. This is manual work you do once with your instructor.

Option B: Upgrade to DKT (Month 3–4)

Once BKT is running, you implement DKT using the PyTorch collection:

```
# From: https://github.com/hcnoh/knowledge-tracing-collec
# DKT uses an LSTM over the sequence of student interacti

# Input per timestep: one-hot(skill_id, correct/incorrect
# Output: predicted probability of correctness for each s

# Your training data is a sequence per student:
```

```
# [(skill_1, correct), (skill_2, incorrect), (skill_3, co
```

The key difference: BKT treats each skill independently. DKT learns dependencies between skills (e.g., understanding loops helps with recursion). DKT gives you a hidden state vector that represents the student's overall knowledge state — this is what you'll inject into the LLM prompt.

Option C: Your Research Contribution — Telemetry-Enriched DKT (Month 4–6)

This is where you go beyond existing work. Standard DKT only gets binary input: correct/incorrect per submission. You enrich it with signals from IDE telemetry:

```
Standard DKT input per timestep:
  [skill_id, correct]

Your enriched DKT input per timestep:
  [skill_id, correct, time_spent, num_compilations, num_e
   error_type_distribution, edit_distance_from_previous,
   copy_paste_detected, idle_time_before_submission]
```

This is a straightforward extension — you concatenate additional features to the input embedding. The research question is whether these extra signals improve prediction accuracy.

How the KT Model Updates in Real Time

You do NOT retrain the model during the lab. Here's the actual flow:

1. **Before the semester**: Train the model on historical data (either from the CSEDM public dataset for initial development, or from your own lab's past semester data if available).
2. **During the lab**: The trained model does inference only. As each student submits, you feed the new (skill, correct, features) tuple

through the model, which updates that student's hidden state (knowledge vector). This is just a forward pass through the LSTM — it takes milliseconds.

3. **The knowledge vector is the output**: A vector like [0.9, 0.3, 0.1, 0.7, ...] where each dimension corresponds to estimated mastery of a concept. This vector is what gets injected into the LLM prompt.

What If You Don't Have Historical Data?

This is a real concern. Options:

Option 1 (Recommended): Start with the CSEDM 2021 dataset (130K+ submissions from 800+ students on intro programming problems, publicly available at https://sites.google.com/ncsu.edu/csedm-dc-2021/). Train your model on this data. Then, when you deploy in your FAST lab, the model will at least have a reasonable prior. It won't be perfectly calibrated to your course, but it will know that "a student who fails recursion problems usually also struggles with trees."

Option 2: Run a "cold start" semester where you just collect data without providing adaptive hints (students get a basic CodeHelp-style assistant). Use that semester's data to train the model. Then in the next semester, deploy the full system. This is cleaner scientifically but takes two semesters.

Option 3: Use the LLM itself for initial concept assessment. At the start of each lab, give students a short diagnostic quiz (3–5 questions). Use the LLM to grade/analyze responses and initialize the knowledge vector. Then update it as the lab progresses. This is the approach the DPKT paper takes — using LLMs to assess difficulty and knowledge.

PART 4: The LLM — Fine-Tuning vs. Prompting

You Probably Don't Need to Fine-Tune

This is critical to understand: **fine-tuning an LLM is almost certainly NOT the right approach for your FYP**. Here's why:

The problem: You want the LLM to give hints, not solutions. You want it to adjust hint level based on the student's knowledge state. You want it to be Socratic.

Why fine-tuning is wrong:

- You need a dataset of (student_code, ideal_Socratic_hint) pairs to fine-tune. You don't have this and can't create it at scale.
- Every existing system in the literature (CodeHelp, CodeAid, STAP, ClassAid) uses **prompting**, not fine-tuning.
- Fine-tuning is expensive, fragile, and a research project in itself.

What to do instead: Structured prompting with an open-weight model.

The Prompting Architecture

```
SYSTEM PROMPT:
You are a Socratic programming tutor for a Data Structure

RULES:
- NEVER provide complete code solutions
- NEVER provide more than {hint_level} level of disclosur
- Adjust your language to the student's demonstrated skil

STUDENT KNOWLEDGE STATE (from KT model):
- Arrays: MASTERED (0.92)
- Linked Lists: DEVELOPING (0.61)
- Recursion: STRUGGLING (0.23)
- Trees: NOT INTRODUCED (0.05)

CURRENT PROBLEM:
{problem_description}

STUDENT'S CURRENT CODE:
```

```
{student_code}

STRUGGLE SIGNALS:
- 3 compilation errors in last 5 minutes (all: missing ba
- Edit pattern: repeatedly modifying the same function
- No copy-paste detected

HINT LEVEL: {determined_by_policy}
Level 0 = Conceptual question only ("What happens when n
Level 1 = Strategy direction ("Your function needs to han
Level 2 = Targeted explanation of the weak concept
Level 3 = Pseudocode skeleton with blanks
Level 4 = Partial code with one key piece missing

Based on the student's knowledge state and struggle signa
a Level {hint_level} hint about their specific issue.
```

Which Open-Weight Model to Use

For deployment in your FAST NUCES lab:

Best option: Qwen2.5-Coder (7B or 14B)

- Open weights, specifically trained for code understanding
- Runs on a single GPU (even a 3090/4090 for the 7B version)
- Strong at understanding code structure and generating explanations
- Can run locally — no API costs, no internet dependency during lab

Alternative: CodeLlama (7B or 13B)

- Meta's code-focused LLM, open weights
- Well-documented, lots of community support
- Slightly older but very stable

Alternative: Llama 3.1 (8B)

- More general but very capable

- Strong instruction-following for the Socratic style

For development/testing: Use Claude or GPT-4 API

- Much easier to prototype with during development
- Switch to local model for deployment

If You Still Want to Fine-Tune (Advanced, Optional)

If your supervisor specifically wants fine-tuning as a research component:

How to get a dataset:

1. Use an existing tutoring dialogue dataset like MathDial or CoMTA (available through the Dialogue-KT repo)
2. Generate synthetic training data: Use GPT-4/Claude to generate (code, ideal_hint) pairs at each disclosure level for your course's problems. Have your instructor validate a sample.
3. Collect real data during your cold-start phase: Log all student-AI interactions, have your instructor rate each hint on a scale, and use the good ones for fine-tuning.

What to fine-tune on: Not the whole model. Use LoRA (Low-Rank Adaptation) on Qwen2.5-Coder-7B. This is cheap (trainable on a single GPU in hours), reversible, and effective for style/behavior changes. Libraries: Hugging Face PEFT + TRL.

But again — prompting-only is the standard approach in the literature and is likely sufficient.

PART 5: Do You Need a Controlled Study? Yes.

Your supervisor said 50% research. Research without evaluation is not research. You need empirical evidence that your system works better

than the alternative. Here's how to do it practically:

Study Design (Realistic for FAST NUCES)

Participants: One section of a Data Structures (or similar) course. Ideally 40–60 students. Ask your supervisor to arrange with a course instructor.

Design: Within-subjects crossover (more practical than between-subjects with your sample size):

- Week 1–3: All students use **System A** (basic reactive assistant, CodeHelp-style — no KT, no proactive hints)
- Week 4–6: All students use **System B** (full Scaffold — with KT-driven adaptive hints)
- Or alternate: Half the class gets A then B, other half gets B then A

What to measure:

1. **Task completion rate** — do students finish lab problems?
2. **Time to correct solution** — are they faster with Scaffold?
3. **Hint usage patterns** — how many hints needed? Do they decrease over time?
4. **Transfer performance** — give an unseen problem WITHOUT any AI help at the end of each phase. Do Scaffold students perform better?
5. **Code quality** — structural quality of final submissions (cyclomatic complexity, code style)
6. **Student survey** — perceived usefulness, learning, frustration (use a standard instrument like SUS or a custom Likert scale)
7. **Copy-paste rate** — does Scaffold reduce solution copying?

What to compare (your paper's results table):

Metric	Reactive Assistant (no KT)	Scaffold (with KT)	p-value
Avg. completion rate	?	?	?

Avg. time to solution	?	?	?
Hints per problem	?	?	?
Transfer task score	?	?	?
Copy-paste incidents	?	?	?

Ethics: You need IRB/ethics approval from FAST. Start this process early (Month 2) — it often takes 4–6 weeks. The study involves collecting student interaction data, which requires informed consent.

PART 6: Realistic Month-by-Month Timeline

Semester 1: Research + Foundation Building

Month 1 (September): Literature & Proposal

- Read the 15 core papers listed above (focus on CodeHelp, Code-DKT, STAP, ClassAid)
- Write the FYP proposal with your research questions
- Define the concept taxonomy for your target course
- Start IRB/ethics application
- Deliverable: Written proposal approved by supervisor

Month 2 (October): Data Exploration + BKT Baseline

- Download and explore the CSEDM 2021 dataset
- Install pyBKT, run BKT on CSEDM data, understand the outputs
- Design your telemetry event schema (what events to capture from the IDE)
- Start building the web-based code editor (Monaco Editor embedded in React/Next.js)

- Deliverable: Working BKT model on CSEDM data + basic code editor prototype

Month 3 (November): DKT Implementation + Telemetry Pipeline

- Implement DKT using the PyTorch collection on CSEDM data
- Compare BKT vs DKT prediction accuracy (AUC) — this is your first research result
- Build the telemetry capture module in the code editor (keystroke logging, compilation events, copy-paste detection)
- Deliverable: BKT vs DKT comparison table + editor with telemetry

Month 4 (December): Telemetry-Enriched DKT + Struggle Detector

- Extend DKT with telemetry features (your novel contribution)
- Build the rule-based struggle detector (repeated same error, long idle, thrashing, regression)
- Test with synthetic data / have classmates use the editor and collect pilot data
- Deliverable: Enriched DKT model + working struggle detector

Month 5 (January): LLM Integration + Hint Engine

- Build the structured prompt system (knowledge state + struggle signals → hint prompt)
- Set up the open-weight model (Qwen2.5-Coder) for local inference, or use API during development
- Implement the disclosure level policy (which hint level based on KT state + struggle duration)
- Test hint quality manually — have your instructor evaluate 50 generated hints
- Deliverable: End-to-end hint generation working (code → KT → prompt → hint)

Month 6 (February): Mid-Year Review / Integration Testing

- Integrate all components into one running system
- Build the output filter / guardrail layer (detect code in LLM response, jailbreak detection)
- Build basic instructor dashboard
- Internal testing with your team + 5–10 volunteers

- Deliverable: Complete working prototype, mid-year FYP presentation

Semester 2: Deployment + Evaluation + Thesis

Month 7 (March): Hardening + Pilot Test

- Fix bugs from internal testing
- Run a 1-week pilot with a small group (10–15 students from your batch)
- Collect feedback, iterate on UI and hint quality
- Finalize study protocol and get ethics approval confirmed
- Deliverable: Pilot test report, finalized study design

Month 8 (April): Classroom Deployment Phase 1

- Deploy System A (reactive assistant without KT) for 2–3 weeks
- Collect all interaction data, baseline metrics
- Monitor system stability
- Deliverable: Phase 1 data collected

Month 9 (May): Classroom Deployment Phase 2

- Switch to System B (full Scaffold with KT-driven hints)
- Deploy for 2–3 weeks
- Administer transfer tasks and surveys
- Deliverable: Phase 2 data collected

Month 10 (June): Analysis + Additional Experiments

- Statistical analysis of Phase 1 vs Phase 2 results
- Run the enriched-DKT vs standard-DKT comparison on your collected data (research result)
- Analyze hint effectiveness, leakage incidents, copy-paste reduction
- Deliverable: All results tables and figures

Month 11 (July): Thesis Writing

- Write the full thesis: Introduction, Literature Review, System

Design, Implementation, Evaluation, Discussion, Conclusion
- Prepare the final FYP presentation
- Deliverable: Thesis draft

Month 12 (August): Defense Preparation

- Revise thesis based on supervisor feedback
- Prepare defense presentation and demo
- Deliverable: Final thesis + defense

PART 7: Where the Motivation Comes From

Every month has a concrete deliverable. Here's what keeps you going:

- **Month 2**: You see a knowledge tracing model actually predicting whether a student will get the next problem right. That's tangible. "The model says this student has 0.23 mastery of recursion." That's cool.

- **Month 3**: You type code in your custom editor and see every keystroke logged in real time. You can replay a student's coding session. That feels powerful.

- **Month 5**: You feed a student's struggling code + their knowledge profile into the LLM, and it generates a hint that says "Think about what happens when your list has only one element" instead of giving the answer. That's the core product working.

- **Month 6**: You demo the full system to your supervisor. Student types code → system detects struggle → system generates a KT-calibrated hint → student solves the problem. That's the "wow moment."

- **Month 9**: You have real data from real students showing that your system actually helped them learn better. That's the research payoff.

The core motivating insight: You're not building a chatbot. You're

answering a question nobody has answered: "Does a tutor that knows what you know give better hints than a tutor that doesn't?"

PART 8: Honest Assessment of Risks

Risk 1: You can't get classroom access for the study. Mitigation: Start talking to instructors NOW. Your supervisor should help arrange this. Worst case, test with your own batch instead of a formal course.

Risk 2: The KT model doesn't improve prediction much over a simple baseline. Mitigation: Even a negative result is publishable if well-documented. "Telemetry features did not significantly improve DKT on our dataset" is a valid finding. But also, BKT-based mastery estimates might still be useful for hint calibration even if prediction accuracy is modest.

Risk 3: Students don't use the system, or try to jailbreak it constantly. Mitigation: Make the system the only help available during lab (instructor agreement). Log all jailbreak attempts — they become data for your guardrail evaluation.

Risk 4: Open-weight LLM generates poor hints. Mitigation: Start development with Claude/GPT-4 API (higher quality). Only switch to local model for deployment. If local model quality is too low, that itself is a finding — and you can discuss the quality-cost-privacy tradeoff in your thesis.

Risk 5: Scope creep — you try to build everything and finish nothing. Mitigation: The MVP is: code editor + BKT mastery tracking + structured LLM prompting with mastery injection. If you get this working and evaluated, you have a thesis. Everything else (DKT, telemetry enrichment, fancy dashboard, guardrail fine-tuning) is bonus.